pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Introduction to Python, Part 1

WEEK 1, DAY 1, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of this part you can

Store a bond

Its ticker, coupon, price, and what you hold.

```
coupon = 5.25
```

Ask a portfolio questions

Totals, averages, highest and lowest.

```
sum(face_list)
```

Work with text

Pull an identifier apart. Clean up a rating.

```
rating_text.strip().upper()
```

Work safely with AI

Read, predict, run, and check what it writes.

```
sum(face_list)  # is this right?
```

One desk, four bonds

Ticker	Issuer	Coupon	Price	Rating
QVNE	Quorvane Energy	5.25	98.50	BBB
DNMR	Dunmarrow Rail	6.125	103.25	BB+
PLWK	Pellwick Paper	4.75	95.00	A-
HLVF	Halvering Foods	7.50	101.00	B+

Every issuer in this course is invented.

How every notebook works

Q.
A question the desk would ask

↓

Code
A few lines that answer it

↓

Takeaway
What the output tells us

A variable is a name for a value

Name on the left, value on the right

```
ticker = 'QVNE'
coupon = 5.25
maturity = 2033
price = 98.50
```

From then on, use the name and Python fills in the value.

Four kinds of value

`str`	Text, always in quotes	`'QVNE'`
`float`	A number with a decimal point	`5.25`
`int`	A whole number	`2033`
`bool`	True or False	`True`

Percent means divide by 100

Right

```
annual_income = face * coupon / 100
$52,500.00
```

Wrong, and Python raises no error

```
annual_income = face * coupon
$5,250,000.00
```

Know roughly what the answer should be before you run it.

Mars Climate Orbiter, 1999

A ground software file supplied thruster data in English units. The trajectory models expected metric.

Both numbers were valid. The units did not match. Nothing raised an error.

The spacecraft arrived about 170 kilometers lower than planned.

Source: Mars Climate Orbiter Mishap Investigation Board, Phase I Report, November 10, 1999.

Format codes, and how a notebook runs

An f-string format code changes what is shown, not the value

Python	Excel	Shows
`f'{annual_income:,.2f}'`	`=TEXT(A1,"#,##0.00")`	52,500.00
`f'{price:.2f}'`	`=TEXT(A1,"0.00")`	98.50
`f'{coupon:.3f}'`	`=TEXT(A1,"0.000")`	5.250
`f'{0.0525:.2%}'`	`=TEXT(A1,"0.00%")`	5.25%

In Excel

A format code is Excel's number format. TEXT() builds text from a number, and & joins the pieces. An f-string does both in one line.

How a notebook runs

A value, not a formula

cost stays 985,000.00 when face changes. Excel recalculates. Python never does.

Run from top to bottom

Each cell relies on the cells above it. Run a cell twice and its work is done twice.

When in doubt, start again

Restart clears what Python remembers. Run All runs every cell in order.

Every arithmetic operator, next to Excel

Python	What it does	Excel
`+  -  *  /`	add, subtract, multiply, divide	`+  -  *  /`
`2350000 // 1000000`	divide and round down: 2	`=QUOTIENT(...)`
`2350000 % 1000000`	the remainder: 350000	`=MOD(...)`
`1.025 ** 10`	raise to a power	`=1.025^10`
`total += payment`	add to a variable	`(no operator)`
`round(2.5)`	halfway goes to even: 2	`=ROUND(2.5,0)` gives 3

^ is not a power

```
2 ^ 3
1
```

No error, wrong answer. Use ** instead.

Brackets first

Then **, then * / // %, then + -. Excel's -2^2 is 4. Python's -2 ** 2 is -4.

Three ways to hold more than one value

List [ ]

One column of a sheet. Ordered, and it can be changed.

```
coupon_list =
  [5.25, 6.125, 4.75, 7.5]
```

A column

Tuple ( )

Like a list, but it cannot be changed after it is made.

```
frequencies = (1, 2, 4)
```

Something fixed

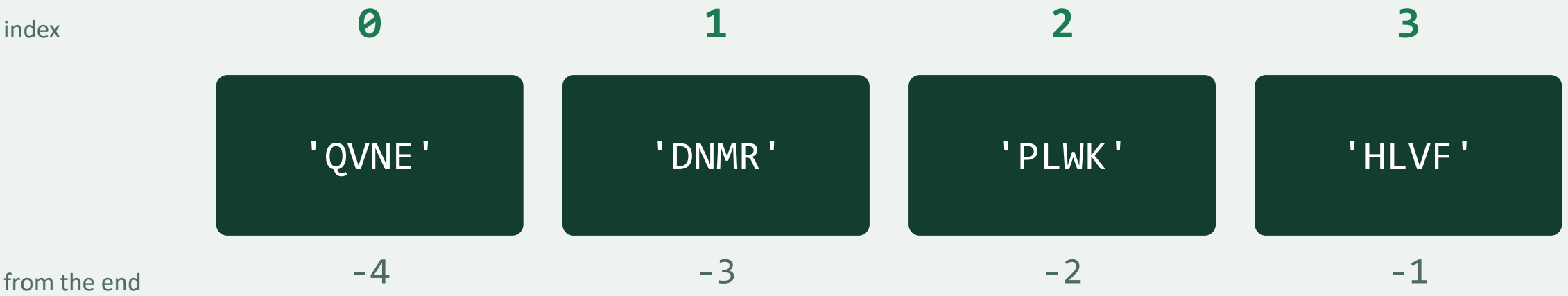
Dictionary { }

One row of a sheet. Each value has a name, called a key.

```
bond = {'Ticker': 'QVNE',
        'Coupon': 5.25}
```

A record

Python counts from zero



The first item is index 0. The last item is always index -1.

Python	`len(list)`	`sum(list)`	`min(list)`	`max(list)`
Excel	`COUNTA`	`SUM`	`MIN`	`MAX`

A string works like a list of characters

Indexing, slicing, and len() all work on text, and every line has an Excel form

Python	Excel
`len(cusip)`	`=LEN(A1)`
`cusip[:6]`	`=LEFT(A1,6)`
`cusip[6:8]`	`=MID(A1,7,2)`
`cusip[-1]`	`=RIGHT(A1,1)`
`rating_text.strip()`	`=TRIM(A1)`
`rating_text.strip().upper()`	`=UPPER(TRIM(A1))`
`face_text.replace(',', '')`	`=SUBSTITUTE(A1,",","")`

cusip is '99012QAB6'. MID counts from 1, so its 7 is Python's index 6.

Working with an AI assistant

What you type into an AI tool or upload to a cloud notebook leaves your machine. A stricter firm policy wins.

Never put into an AI tool or a cloud notebook	Fine to use
Client and counterparty names	The synthetic data in this course
Positions, holdings, and trades	Public data, such as published Treasury yields
Anything from your employer: files, code, emails	Code you wrote yourself on that data
Passwords, keys, and other credentials	

How to ask

Say what you have, what you want, and the units.

I have a list of face amounts in dollars and a list of prices per 100 of face. Write code that totals the cost in dollars.

How to read what comes back

Read. Predict. Run. Check.

Code that runs without an error can still be wrong. The last exercise of every notebook is an AI check.

What you can do now

- Store a bond in variables and calculate with it
- Hold a portfolio in lists and dictionaries, and total it
- Pull a string apart and clean it up
- Decide what may go into an AI tool, and check what comes back

Next Day 1, Part 2: decisions, loops, and functions on the same four bonds